

Relatório Técnico: Classificação de Imagens com Perceptron Simples

Pós-Graduação em Inteligência Artificial Aplicada

Disciplina: Machine Learning

Professor: Dr. Sirlon Diniz de Carvalho

Aluno: Flávio Lourenço da Silva

Atividade: Classificação de Imagens (Cães vs. Gatos)

1. Introdução

O presente relatório descreve o desenvolvimento e a fundamentação teórica de um sistema de classificação binária de imagens (cães versus gatos) utilizando o modelo clássico do **Perceptron de Rosenblatt (1958)**. A arquitetura do Perceptron representa o marco inicial da era das redes neurais artificiais, sendo o primeiro modelo computacional capaz de aprender pesos sinápticos a partir de exemplos de treinamento através de uma regra de aprendizado supervisionado.

O objetivo do projeto consiste em:

- Implementar um fluxo de processamento de imagens brutas para conversão em vetores numéricos adequados;
- Implementar a estrutura matemática e a regra de atualização de pesos do Perceptron simples;
- Avaliar a eficácia do classificador linear sobre um conjunto de dados complexo e não-linearmente separável (cães vs. gatos).

2. Processamento e Normalização dos Dados

As imagens digitais são armazenadas originalmente como matrizes tridimensionais (altura, largura e três canais de cor RGB). Para alimentar um modelo simples como o Perceptron, é necessário criar um pipeline de pré-processamento que reduza a dimensionalidade dos dados sem perder informações estruturais cruciais.

O fluxo de processamento foi implementado no arquivo *normalizacao.py* e fundamenta-se nos seguintes passos:

2.1 Conversão para Escala de Cinza

A conversão da imagem colorida para escala de cinza (canal único 'L') é realizada pela biblioteca PIL (Pillow) através da fórmula de luminância: $Y = 0.299R + 0.587G + 0.114B$. Essa transformação reduz a quantidade de dados em 66,7%. De acordo com **Gonzalez e Woods (2018)**, a cor muitas vezes atua como um ruído redundante em classificações morfológicas simples. A escala de cinza preserva características estruturais fundamentais, como formas, bordas e intensidades de textura, que são suficientes para a modelagem estatística primária.

2.2 Redimensionamento (Resizing)

As imagens do dataset possuem resoluções variadas, impossibilitando sua inserção direta em uma camada de entrada estática. Todas as imagens foram redimensionadas para uma resolução fixa de 32x32 pixels. O tamanho resultante de 1024 pixels atua como um compromisso ideal entre a preservação de detalhes visuais mínimos e a mitigação da '**maldição da dimensionalidade**' (Bellman, 1957). Resoluções maiores exigiriam um número excessivo de parâmetros (pesos), levando a problemas de sobreajuste (*overfitting*) e alta latência computacional.

2.3 Normalização de Intensidades (Min-Max Scaling)

Os valores originais dos pixels variam no intervalo discreto de [0, 255]. A normalização é aplicada dividindo cada pixel por 255.0, escalando os valores para o intervalo contínuo [0.0, 1.0]. A normalização é uma etapa crítica de preparação de dados para algoritmos baseados em descida de gradiente ou regras de aprendizagem iterativas. Conforme destacado por **Gonzalez e Woods (2018)**, o escalonamento de atributos (*feature scaling*) evita a dominância numérica de certos pixels sobre outros, previne a saturação precoce de funções de ativação e garante a estabilidade numérica durante o ajuste dos pesos sinápticos, acelerando a convergência.

2.4 Vetorização (Flattening)

A matriz de pixels normalizada de dimensão 32x32 é convertida em um vetor unidimensional contíguo de 1024 atributos. Ao final do vetor, é anexado o rótulo da classe correspondente (0 para gato, 1 para cão), que servirá como alvo (*target*) durante o treinamento supervisionado. Os dados processados são então exportados e gravados em arquivos CSV na pasta *dataset_normalizado*.

2.5 Particionamento dos Dados (Holdout 80/20)

Após a vetorização de todas as imagens, o conjunto completo de amostras é **embaralhado aleatoriamente** (`random_state=42` para garantir reprodutibilidade) e dividido em dois subconjuntos mutuamente exclusivos:

- **Treino (80%)**: Utilizado para ajustar os pesos sinápticos do Perceptron durante as épocas de aprendizado. Composto por **1.600 amostras**.
- **Teste (20%)**: Mantido completamente isolado durante o treinamento, sendo utilizado apenas para avaliar a capacidade de generalização do modelo com dados inéditos. Composto por **400 amostras**.

Essa estratégia, conhecida como *Holdout*, é a abordagem mais simples de validação em aprendizado de máquina e garante que a métrica de acurácia reportada reflita o desempenho real do modelo sobre dados não vistos, evitando a estimativa otimista que ocorreria ao avaliar o modelo sobre os próprios dados de treino (**Hastie, Tibshirani e Friedman, 2009**).

3. Arquitetura e Formulação Matemática do Perceptron

O modelo implementado no arquivo *perceptron.py* segue rigorosamente a especificação do neurônio artificial proposto por **Frank Rosenblatt (1958)**.

3.1 Função de Integração (Soma Ponderada)

O potencial de ativação u do neurônio é calculado pela soma ponderada dos valores de entrada multiplicados por seus respectivos pesos sinápticos, adicionando-se o termo de polarização (*bias*):

$$u = \sum (w_j * x_j) + \theta$$

Onde x_j representa a intensidade normalizada do pixel j , w_j representa o peso sináptico associado ao pixel j (inicializado aleatoriamente no intervalo $[-0.5, 0.5]$) e θ representa o *bias* (limiar de ativação).

3.2 Função de Ativação (Degrau Heaviside)

A saída do classificador $\hat{y}$ é gerada pela aplicação da função degrau unitário sobre o potencial de ativação u :

$$\hat{y} = g(u) = 1 \text{ se } u \geq 0 \text{ else } 0$$

Onde 1 representa a classe positiva (cão) e 0 a classe negativa (gato).

3.3 Regra de Aprendizado

Durante a fase de treinamento, os pesos e o bias são ajustados iterativamente a cada exemplo apresentado, com base no erro de previsão. O erro é a diferença entre o rótulo real y e o previsto $\hat{y}$: $E = y - \hat{y}$. A atualização de cada peso w_j e do bias θ é regida pelas fórmulas:

$$\begin{aligned} w_j(t+1) &= w_j(t) + \eta * E * x_j \\ \theta(t+1) &= \theta(t) + \eta * E \end{aligned}$$

Onde η é a taxa de aprendizado, configurada em 0.01. Ela controla a magnitude dos ajustes nos parâmetros a cada iteração. Se $E = 0$ (previsão correta), nenhuma alteração ocorre. Se $E \neq 0$, os pesos são deslocados na direção que minimiza o erro futuro para aquela entrada específica.

4. Análise de Limitações Teóricas e Discussão

4.1 O Teorema de Convergência do Perceptron

O Teorema de Convergência do Perceptron, demonstrado por **Rosenblatt (1958)**, afirma que se os dados de treinamento de duas classes forem **linearmente separáveis** (isto é, se existir um hiperplano no espaço de entrada que separe perfeitamente as duas classes), o algoritmo de treinamento garantidamente convergirá para uma solução com erro zero em um número finito de épocas.

4.2 O Problema da Não-Separabilidade Linear

Em 1969, **Marvin Minsky e Seymour Papert** publicaram o livro *Perceptrons*, demonstrando matematicamente que o Perceptron de camada única é incapaz de resolver problemas que não sejam linearmente separáveis (sendo o exemplo clássico a porta lógica XOR).

A classificação de imagens complexas (como cães vs. gatos) a partir de pixels brutos normalizados é um problema altamente não-linear por diversas razões:

1. **Morfologia Complexa:** A diferença visual entre cães e gatos não está correlacionada a intensidades simples de pixels locais, mas sim a padrões hierárquicos e espaciais (formato de orelhas, focinho, textura do pelo).
2. **Variações de Ambiente:** Variações na pose do animal, condições de iluminação, rotação, zoom e presença de objetos em segundo plano alteram drasticamente os valores dos pixels, de modo que os espaços de características de ambas as classes se sobrepõem de forma complexa no espaço de 1024 dimensões.
3. **Ausência de Extrator de Recursos:** Diferente de arquiteturas modernas, o Perceptron recebe os pixels de forma independente, sem considerar a correlação espacial local entre os pixels vizinhos.

Portanto, o erro de treinamento do Perceptron sobre o dataset dificilmente atingirá zero e tenderá a oscilar após algumas épocas. Esse comportamento justifica teoricamente o uso de arquiteturas multicamadas (MLP) com funções de ativação não-lineares combinadas com algoritmos de retropropagação do erro (*Backpropagation*), ou Redes Neurais Convolucionais (CNNs), que utilizam filtros espaciais para extrair recursos hierárquicos antes da classificação linear final.

5. Resultados Experimentais

O experimento foi executado com o conjunto de dados completo particionado conforme a estratégia Holdout 80/20 descrita na Seção 2.5. Os parâmetros de treinamento utilizados foram:

- **Amostras de Treino:** 1.600
- **Amostras de Teste:** 400
- **Dimensão do vetor de entrada:** 1.024 pixels
- **Taxa de Aprendizado (η):** 0,01
- **Número de Épocas:** 100

Ao longo das 100 épocas de treinamento, o número de erros por época apresentou tendência decrescente com oscilação característica — partindo de **777 erros** na Época 1 e chegando a **482 erros** na Época 100 — comportamento típico de dados não-linearmente separáveis, onde o algoritmo nunca estabiliza em erro zero.

O resultado final de avaliação sobre o conjunto de **400 amostras de teste** (nunca vistas durante o treinamento) foi:

- **Acertos:** 210
- **Total de Amostras:** 400
- **Acurácia:** 52,50%

A acurácia de **52,50%** é marginalmente superior ao limiar de decisão aleatória de 50%, o que confirma empiricamente a incapacidade estrutural do Perceptron simples de extrair padrões discriminativos suficientes para a separação não-linear entre as classes de cães e gatos a partir de pixels brutos normalizados — resultado plenamente consistente com as limitações teóricas discutidas na Seção 4.

6. Estrutura do Projeto Desenvolvido

O código foi segmentado de forma modular e limpa para atender aos critérios de engenharia de software e legibilidade:

1. **normalizacao.py:** Responsável pela leitura de todas as imagens do diretório *dataset/*, conversão para escala de cinza, redimensionamento (32x32), normalização Min-Max, embaralhamento aleatório do conjunto completo e particionamento Holdout 80/20 — gerando os arquivos *treino.csv* (1.600 amostras) e *teste.csv* (400 amostras) dentro do diretório *dataset_normalizado/*.
2. **perceptron.py:** Contém a definição da classe *Perceptron* com os métodos construtor, *soma_ponderada*, *ativacao*, *prever* e *treinar*.
3. **main.py:** Orquestra o pipeline completo. Carrega os arquivos CSV normalizados, inicializa o Perceptron com 1.024 entradas, executa o treinamento ao longo de **100 épocas** exibindo o número de erros a cada ciclo, realiza o teste com dados não vistos e reporta a acurácia final obtida.

7. Referências Bibliográficas

- **GONZALEZ, Rafael C.; WOODS, Richard E.** *Digital Image Processing*. 4th ed. New York: Pearson Education, 2018.
- **HASTIE, Trevor; TIBSHIRANI, Robert; FRIEDMAN, Jerome.** *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York: Springer, 2009.
- **MINSKY, Marvin; PAPERT, Seymour.** *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969.
- **ROSENBLATT, Frank.** The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, v. 65, n. 6, p. 386–408, 1958.